

Taller NoSQL

Integrantes:

- Alexis Romero - CI 4.653.441-7
- Andrés Fernández - CI 4.859.574-0
- Dámaso Tor - CI 4.508.724-7
- Mateo Sparano - CI 5.545.828-2

1. Realidad en la cual utilizaremos MongoDB

1.1 Descripción general

Proponemos utilizar un sistema de base de datos no relacional con MongoDB para implementar un e-commerce de una tienda que vende diversos productos.

Creemos que una base de datos no relacional tiene algunas ventajas para un proyecto así.

Flexibilidad del esquema

La tienda venderá artículos diferentes. Y debe mostrar información a los visitantes con datos que son diferentes entre las distintas categorías de productos.

En ese sentido, en la descripción de una camiseta, tendremos los datos de talla, color y marca. En cambio, en otros tipos de productos como muebles, tendremos características como altura, largo, profundidad, material.

Y a futuro, la tienda podría expandir su catálogo a otros rubros, sin necesidad de cambiar el esquema de la base de datos.

Por tanto, la flexibilidad del esquema que tiene MongoDB permitiría guardar para cada categoría de productos los atributos que necesita realmente, sin la complicación y los grandes JOIN que serían necesarios en una base de datos relacional para hacerlo.

Escalado horizontal

Los e-commerce experimentan cargas de trabajo impredecibles, con picos y valles de acuerdo a la afluencia de los compradores. El soporte nativo de MongoDB para el escalado horizontal permite afrontar este reto con soluciones en la nube que permiten agregar y

reducir servidores de base de datos de acuerdo a la demanda, sin el costo de los recursos que se necesitan en los picos durante todo el tiempo.

Las bases de datos que solo soportan escalado vertical requiere contratar servidores de alto desempeño durante toda su operación.

Este escalado horizontal permite también fortalecer la alta disponibilidad de la base de datos, evitando un punto único de falla. De esta manera, evitamos fallas o saturaciones en picos extremos de demanda, que causan downtime.

Las bases NoSQL priorizan la disponibilidad sobre la consistencia estricta, lo que es ideal para lecturas rápidas en e-commerce, donde un pequeño retraso puede perder ventas.

Estructura de datos

La estructura de datos documental de MongoDB se alinea mejor con el modelado de objetos en nuestro código. Esto nos permite representar entidades complejas, como un cliente con múltiples direcciones y pedidos, en un solo documento, simplificando las consultas y mejorando el rendimiento al evitar JOINS complejos.

Tipo de consultas

MongoDB además de ofrecer operaciones atómicas para agregar, eliminar o actualizar datos en una colección, también ofrece el Aggregation Pipeline, que nos permitirá realizar búsquedas avanzadas de productos. Esto nos habilita a ofrecer al visitante búsquedas avanzadas de acuerdo a criterios complejos.

1.2 Casos de uso

1.2.1 Filtro avanzado de productos

Actor: Visitante

Descripción: El visitante desea filtrar los productos por distintas características o especificaciones de los productos. Asimismo, se podrá filtrar por sucursales en las que haya stock del artículo.

Ventajas de utilizar MongoDB:

La estructura de datos documental permite concentrar en un documento los datos que se requieren del producto, incluyendo arreglos con sucursales donde hay determinado stock. Esto nos evita unir tablas, que es una operación costosa en grandes cantidades de datos. Las consultas de MongoDB permiten condiciones complejas sobre campos anidados y elementos dentro de arreglos de forma nativa.

ID CU:	CU-001
Nombre CU:	Filtro avanzado de productos
Actor principal:	Visitante
Objetivo:	Permitir al Visitante aplicar filtros a la lista de productos visibles, basándose en características, especificaciones y disponibilidad de stock en sucursales.
Precondiciones:	El Visitante está navegando en la sección productos del sitio. Existen productos cargados en el sistema.
Postcondiciones:	El Sistema muestra una lista de productos actualizada y acotada a los productos que cumplen con todos los criterios de filtro seleccionados por el Visitante.
Flujo principal:	1. El Visitante ingresa a la vista de productos y ve la lista completa. 2. El Visitante selecciona uno o varios filtros de características o especificaciones (ej. "Color: Rojo", "Volumen: 750ml"). 3. El Visitante selecciona una o varias Sucursales para filtrar por stock. 4. El Visitante presiona el botón "Aplicar Filtros". 5. El Sistema ejecuta una única consulta a la base de datos que aplica las condiciones sobre las características y busca coincidencias dentro del arreglo de inventario por sucursal. Recibe el conjunto acotado de productos. 6. El Sistema actualiza y muestra la lista filtrada de productos al Visitante.
Flujos alternativos:	A.1. Restablecer filtros. (Antes del paso 4 o después del paso 6.) 1. El Visitante presiona el botón "Limpiar filtros". 2. El Sistema elimina todas las selecciones de filtros de Características y Sucursales. 3. El Sistema ejecuta una consulta para obtener el listado completo original de productos. 4. El Sistema actualiza la vista mostrando el catálogo completo.
Flujos de excepción:	E.1. No se encuentran resultados. (Durante el paso 5). 1. Se muestra el mensaje: "No se encontraron productos que coincidan con los filtros seleccionados".
Requisitos no funcionales:	1. La consulta de filtrado debe devolver resultados en menos de 2 segundos. (Requisito de rendimiento).

1.2.2 Carrito de compras persistente y autolimpiable

Actor: Visitante

Descripción: Un usuario navega por el sitio y agrega productos a su carrito. El carrito debe recordar estos productos entre visitas (sesiones), manejar la adición y eliminación de productos. Si el visitante no concreta la compra en una determinada cantidad de días, el carrito se autolimpia.

Ventajas de utilizar MongoDB:

Todo el carrito del usuario puede ser un único documento. Agregar, quitar o actualizar la cantidad de un producto es una sola operación atómica sobre ese documento. Esto es extremadamente rápido y reduce la carga en la base de datos, lo cual es crucial durante los picos de tráfico.

Asimismo, MongoDB tiene una funcionalidad nativa llamada índices TTL (Time-To-Live), que permite configurar un índice en un campo de fecha para que MongoDB elimine automáticamente los documentos cuando haya pasado un cierto tiempo.

ID CU:	CU-002
Nombre CU:	Gestionar carrito persistente
Actor principal:	Visitante
Objetivo:	Permitir al Visitante mantener un registro de los productos deseados a través de múltiples sesiones. El Sistema limpia automáticamente el carrito si la compra no se completa en determinada cantidad de días.
Precondiciones:	El Visitante tiene una cookie de sesión (o está logueado) que permite identificar su carrito. El catálogo de productos está disponible.
Postcondiciones:	El carrito del Visitante está actualizado con las modificaciones realizadas (productos/cantidades), y el campo de fecha de última actividad está actualizado.
Flujo principal:	1. El Visitante navega por el sitio. 2. El Visitante pulsa "Añadir al Carrito" en un producto. 3. El Sistema localiza o crea al documento del carrito del Visitante en la base de datos. 4. El sistema realiza una única operación atómica para: a. Añadir el producto (si es nuevo) o incrementar su cantidad (si ya existe). b. Actualizar el campo <i>última_actividad</i> del documento del carrito a la hora actual. 5. El Sistema muestra un mensaje de confirmación y el subtotal actualizado del carrito.
Flujos alternativos:	A.1. Eliminar artículo (Se produce después del paso 5). 1. El Visitante selecciona "Eliminar" en un artículo dentro del carrito. 2. El Sistema realiza una operación atómica para remover el subdocumento del artículo y actualizar el subtotal y la última actividad. 3. El Sistema muestra un mensaje de confirmación y el subtotal actualizado del carrito.
Flujos de excepción:	E.1. Producto agotado (Durante el paso 4). 1. Si el producto no tiene más stock, la operación atómica falla. 2. El Sistema muestra un mensaje de error y el carrito no se modifica.
Requisitos no funcionales:	1. Persistencia de sesión. Los productos del carrito deben persistir por un período definido incluso si el Visitante cierra su navegador o dispositivo. 2. Autolimpieza.

	Si un carrito no registra actividad (no hay adiciones o eliminaciones) durante 30 días (ejemplo), debe ser eliminado automáticamente del sistema.
--	---

2. Diseño de la base de datos en MongoDB

2.1 Colecciones:

2.1.1 Productos

Descripción

Esta colección almacena la información de cada producto en venta. Cada documento representa un producto único, con campos flexibles para adaptarse a diferentes categorías (ropa, accesorios, zapatos). Esto permite esquemas variables sin alterar la estructura global.

Estructura:

```
db.createCollection("productos", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["nombre", "precio", "categoria", "creadoEn", "actualizadoEn"],
      properties: {
        nombre: {
          bsonType: "string",
          minLength: 1,
          description: "Debe ser un string no vacío"
        },
        descripcion: {
          bsonType: "string",
          description: "Debe ser un string si se incluye"
        },
        precio: {
          bsonType: "number",
          minimum: 0,
          description: "Debe ser un número no negativo"
        },
        categoria: {
          bsonType: "string",
          enum: ["camisas", "pantalones", "zapatos", "accesorios"],
          description: "Debe ser una categoría válida"
        },
        atributos: {
          bsonType: "object",
```

```

        description: "Objeto flexible para características específicas (color,
talla, material, etc.)"
    },
    imagenes: {
        bsonType: "array",
        items: {
            bsonType: "string"
        },
        description: "URLs de imágenes del producto"
    },
    inventario: {
        bsonType: "array",
        items: {
            bsonType: "object",
            required: ["sucursal_id", "stock", "ultimaActualizacion"],
            properties: {
                sucursal_id: {
                    bsonType: "string",
                    description: "ID de la sucursal"
                },
                stock: {
                    bsonType: "number",
                    minimum: 0,
                    description: "Stock disponible (no negativo)"
                },
                ultimaActualizacion: {
                    bsonType: "date"
                }
            }
        }
    },
    creadoEn: {
        bsonType: "date",
        description: "Fecha de creación"
    },
    actualizadoEn: {
        bsonType: "date",
        description: "Fecha de última actualización"
    }
}
},
validationLevel: "strict",
validationAction: "error"
});

// Índices para optimización
db.productos.createIndex(
    { "inventario.sucursal_id": 1, "inventario.stock": 1 },
    { name: "idx_inventario_sucursal_stock" }
);
db.productos.createIndex({ "categoria": 1 });
db.productos.createIndex({ "atributos.color": 1 });

```

Explicación

Los campos como "atributos" son objetos dinámicos, permitiendo agregar propiedades nuevas sin migraciones. El arreglo "inventario" embebe el stock por sucursal para consultas rápidas en filtros, evitando joins frecuentes.

Agregamos un índice compuesto entre inventario.sucursal_id e inventario.stock para agilizar consultas del Caso de Uso CU-001, en el que se filtren productos por su stock en determinada sucursal.

2.1.2 Carritos

Descripción

Colección para manejar carritos de compras persistentes por visitante. Cada documento representa un carrito único, identificado por una cookie de sesión o ID de usuario.

Usamos TTL para autolimpieza basada en inactividad.

Estructura:

```
db.createCollection("carritos", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["visitanteId", "registrado", "total", "ultimaActividad"],
      properties: {
        visitanteId: {
          bsonType: "string",
          minLength: 1,
          description: "ID del visitante (cookie o userID)"
        },
        usuarioEmail: {
          bsonType: "string",
          description: "Email del usuario si está autenticado"
        },
        registrado: {
          bsonType: "boolean",
          description: "Si el usuario está registrado o es visitante anónimo"
        },
        items: {
          bsonType: "array",
          items: {
            bsonType: "object",
            required: ["articuloId", "cantidad", "precioUnitario", "subtotal"],
            properties: {
              articuloId: {
                bsonType: "string"
              },
              cantidad: {
                bsonType: "number",
                minimum: 1
              },
              precioUnitario: {
```

```

        bsonType: "number",
        minimum: 0
    },
    subtotal: {
        bsonType: "number",
        minimum: 0
    }
}
},
total: {
    bsonType: "number",
    minimum: 0,
    description: "Total del carrito"
},
ultimaActividad: {
    bsonType: "date",
    description: "Fecha de última actividad para TTL"
}
}
},
validationLevel: "strict",
validationAction: "error"
});

// Índice TTL para autolimpieza (30 días)
db.carritos.createIndex(
    { "ultimaActividad": 1 },
    { expireAfterSeconds: 2592000 }
);
db.carritos.createIndex({ "visitanteId": 1 });

```

Explicación

Todo el carrito se almacena en un solo documento para operaciones atómicas (ej., agregar/quitar items). El campo "ultima_actividad" permite configurar un índice TTL para eliminar carritos inactivos automáticamente después de 30 días, reduciendo el tamaño de la colección con el tiempo.

2.1.3 Sucursales

Descripción

Almacena información de las sucursales físicas, usada para referencias en inventario y filtros por stock.

Estructura:

```

db.createCollection("sucursales", {
    validator: {
        $jsonSchema: {
            bsonType: "object",
            required: ["nombre", "direccion"],
            properties: {

```



```

    nombre: {
      bsonType: "string",
      minLength: 1,
      description: "Nombre de la sucursal"
    },
    direccion: {
      bsonType: "string",
      minLength: 1,
      description: "Dirección completa"
    },
    telefono: {
      bsonType: "string",
      description: "Teléfono de contacto"
    },
    horario: {
      bsonType: "string",
      description: "Horarios de atención"
    },
    activa: {
      bsonType: "bool",
      description: "Si la sucursal está operativa"
    }
  }
},
validationLevel: "strict",
validationAction: "error"
});

```

Explicación

Colección simple y estática, ya que las sucursales no cambian frecuentemente. Sirve como referencia para evitar duplicar datos en "inventario" de productos.

2.1.4 Usuarios

Descripción

Almacena perfiles de usuarios registrados con autenticación y roles.

Estructura:

```

db.createCollection("usuarios", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["email", "passwordHash", "nombre", "rol"],
      properties: {
        email: {
          bsonType: "string",
          pattern: "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$",
          description: "Email válido único"
        },
        passwordHash: {

```

```

        bsonType: "string",
        description: "Contraseña hashada con BCrypt"
    },
    nombre: {
        bsonType: "string",
        minLength: 1,
        description: "Nombre del usuario"
    },
    rol: {
        bsonType: "object",
        required: ["nombre"],
        properties: {
            nombre: {
                bsonType: "string",
                enum: ["ROLE_USER", "ROLE_ADMIN"],
                description: "Rol del usuario"
            }
        },
        description: "Objeto rol del usuario"
    },
    activo: {
        bsonType: "boolean",
        description: "Si el usuario está activo"
    },
    fechaRegistro: {
        bsonType: "date",
        description: "Fecha de registro"
    },
    direcciones: {
        bsonType: "array",
        items: {
            bsonType: "object",
            required: ["calle", "ciudad"],
            properties: {
                calle: {
                    bsonType: "string",
                    minLength: 1
                },
                ciudad: {
                    bsonType: "string",
                    minLength: 1
                }
            }
        }
    }
},
validationLevel: "strict",
validationAction: "error"
});

db.usuarios.createIndex({ "email": 1 }, { unique: true });

```

Explicación

Embebemos direcciones para consultas rápidas.

2.1.5 Órdenes

Descripción

Esta colección almacena las órdenes de compra completadas por los clientes. Cada documento representa una orden única, capturando el estado del carrito en el momento de la compra para mantener la integridad histórica (ej., precios y cantidades fijas, independientemente de cambios posteriores en los productos).

Esto facilita reportes, seguimiento de ventas y análisis de datos, como ventas por sucursal o cliente.

La colección crecerá con el volumen de ventas, pero las operaciones principales son inserts (al finalizar compra) y lecturas para seguimiento.

Estructura:

```
db.createCollection("ordenes", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["usuarioId", "items", "total", "metodoPago", "estado",
"fechaPedido"],
      properties: {
        usuarioId: {
          bsonType: "objectId",
          description: "ID del usuario que realizó la orden"
        },
        visitanteId: {
          bsonType: "string",
          description: "ID del visitante si no está registrado"
        },
        items: {
          bsonType: "array",
          minItems: 1,
          items: {
            bsonType: "object",
            required: ["articuloId", "cantidad", "precioUnitario", "subtotal"],
            properties: {
              articuloId: {
                bsonType: "objectId"
              },
              cantidad: {
                bsonType: "number",
                minimum: 1
              },
              precioUnitario: {
                bsonType: "number",
                minimum: 0
              },
              subtotal: {
                bsonType: "number",
                minimum: 0
              }
            }
          }
        }
      }
    }
  }
})
```

```

    }
  },
  total: {
    bsonType: "number",
    minimum: 0
  },
  sucursalId: {
    bsonType: ["objectId", "null"],
    description: "Sucursal para retiro (opcional)"
  },
  direccionEnvio: {
    bsonType: ["object", "null"],
    properties: {
      calle: { bsonType: "string" },
      ciudad: { bsonType: "string" },
      codigoPostal: { bsonType: "string" }
    }
  },
  metodoPago: {
    enum: ["tarjeta", "efectivo", "transferencia"],
    description: "Método de pago utilizado"
  },
  estado: {
    enum: ["pendiente", "confirmada", "enviada", "entregada", "cancelada"],
    description: "Estado actual de la orden"
  },
  fechaPedido: {
    bsonType: "date"
  },
  fechaActualizacion: {
    bsonType: "date"
  }
}
},
validationLevel: "strict",
validationAction: "error"
});

// Índices para consultas frecuentes
db.ordenes.createIndex({ "usuarioId": 1 });
db.ordenes.createIndex({ "fechaPedido": -1 });
db.ordenes.createIndex({ "estado": 1 });

```

Explicación

Los "items" se embeben como arreglo para operaciones atómicas y consultas rápidas (ej., ver detalles de orden sin joins). Esto captura los detalles de la compra, evitando inconsistencias si un artículo cambia precio o stock después.

Referenciamos "sucursal_id" para asociar con retiros en tienda, "usuario_id" para un seguimiento por cliente, y "articulo_id" para enlazar con productos (usando \$lookup en agregaciones si se necesitan detalles completos).

Campos como "estado" permiten actualizaciones atómicas con \$set.

Para optimización, indexar "usuario_id", "fecha_pedido" y "estado" para consultas frecuentes como órdenes pendientes o ventas mensuales.

2.2 Relaciones entre colecciones:

Datos embebidos:

En Productos:

- atributos como objeto flexible (color, talla, material, etc.).
- inventario como arreglo de subdocumentos con stock por sucursal

Justificación: Las consultas de filtro (CU-001) requieren acceso simultáneo a características y stock, evitando joins costosos.

En Carritos:

- items como arreglo de productos

Justificación: Permite actualizaciones atómicas (agregar/quitar items) en una sola operación.

En Usuarios:

- direcciones como arreglo embebido
- rol como objeto embebido

Justificación: Datos que no cambian frecuentemente y se consultan juntos.

Datos referenciados

- sucursal_id en inventario de Productos → referencia a Sucursales
- articuloId en items de Carritos → referencia a Productos
- usuarioId en Órdenes → referencia a Usuarios
- sucursalId en Órdenes → referencia a Sucursales

Con este diseño obtenemos consultas rápidas utilizando datos embebidos para acceso frecuente, consistencia de datos utilizando referencias para entidades que cambian independientemente, operaciones atómicas con el embebido en carritos y productos y escalabilidad con una estructura preparada para crecimiento.

2.3 Consultas comunes:

2.3.1 Filtro avanzado de productos (CU-001):

Descripción

Buscar productos por categoría, atributos (color, talla) y stock en sucursales específicas.

Consulta MongoDB (Aggregation Pipeline):

```
db.productos.aggregate([
  {
    $match: {
      "categoria": "camisas",
      "atributos.color": "Azul",
      "inventario": {
        $elemMatch: {
          "sucursal_id": "6908e8a0b482ce1ec23cdb83",
          "stock": { $gte: 1 }
        }
      }
    }
  },
  {
    $project: {
      nombre: 1,
      precio: 1,
      categoria: 1,
      atributos: 1,
      imagenes: 1,
      inventario: 1
    }
  }
]);
```

Explicación

\$match filtra eficientemente; \$elemMatch busca en arreglos sin unwind innecesario. Para rendimiento, usar índices en "atributos.color" y "inventario.sucursal_id".

2.3.2 Agregar/Actualizar ítem en carrito (CU-002):

Descripción

Operación atómica para agregar o incrementar cantidad, actualizando total y ultima_actividad.

```

db.carritos.updateOne(
  { "visitanteId": "user_12345" },
  {
    $push: {
      "items": {
        "articuloId": "690aa58b62100689c0eec4a9",
        "cantidad": 1,
        "precioUnitario": 1500.0,
        "subtotal": 1500.0
      }
    },
    $inc: { "total": 1500.0 },
    $set: { "ultimaActividad": new Date() }
  },
  { upsert: true }
);

```

Explicación

Esta consulta guarda el precio unitario al momento de realizarse, para evitar que el valor del carrito se modifique con posibles modificaciones de precios de productos. Asimismo, incrementa el valor de “total” por el subtotal agregado/actualizado. Y actualiza “ultima_actividad” para el control de TTL de 30 días de los carritos.

2.3.3 Autenticación de usuarios:

Buscar usuario por email:

```

db.usuarios.findOne(
  { "email": "usuario@ejemplo.com" },
  { "email": 1, "passwordHash": 1, "rol": 1, "activo": 1 }
);

```

2.3.4 Consulta de inventario por sucursal:

```

db.productos.find(
  { "inventario.sucursal_id": "6908e8a0b482ce1ec23cdb83" },
  { "nombre": 1, "inventario.$": 1 }
);

```

3. Investigación de interacción de MongoDB con lenguajes de programación

Estructura del proyecto:

src/main/java/com/ropa/tienda/

- |— model/ # Entidades MongoDB (Producto, Usuario, Carrito, etc.)
- |— repository/ # Interfaces de repositorio Spring Data
- |— controller/ # Controladores REST API
- |— config/ # Configuraciones (CORS, Security, MongoDB)
- |— security/ # Clases JWT y autenticación
- |— service/ # Servicios de negocio

src/main/resources/

- |— application.properties # Configuración de la aplicación
- |— static/ # Frontend (HTML, CSS, JS)
 - |— index.html # Página principal con catálogo
 - |— login.html # Página de autenticación
 - |— admin.html # Panel administrativo

- Lenguaje elegido:

El proyecto actual está implementado en Java porque es una opción que ofrece portabilidad, escalabilidad y mayor rendimiento. Además utilizamos java con Spring Boot que ayuda mucho a evitar duplicar logica o codigo, aprovecha Spring Data MongoDB y facilita la integración con seguridad (JWT, BCrypt).

- Librerías / drivers

- MongoDB Java Driver.
- Usamos Spring Data MongoDB proporciona repositorios, mapeo de documentos y conveniencia con Spring Boot.

- Ventajas de usar MongoDB con Java / Spring Data

Integración nativa con Spring Boot (autoconfiguración, repositorios, conversores).

Buen rendimiento en operaciones agregadas y flexibles con Aggregation Framework (pipeline).

Tipado en Java para modelos de dominio (clases model/*.java) y validación a nivel de aplicación.

Comunidad amplia y abundante documentación.

Fácil uso de transacciones (cuando aplica) y soporte de índices desde código o scripts.

- Limitaciones / desventajas
 - Esquema flexible → posible inconsistencias si hay conversiones personalizadas o datos previos (ej.: rol guardado como string vs. objeto).
 - Manejo de relaciones complejas (joins) no es tan directo como en RDBMS; requiere \$lookup o desnormalización.

4. Casos de prueba

4.1 Pruebas del CU-001 (Filtro avanzado):

CP-001: Filtrar por categoría "camisas" → Debe retornar 33 productos

CP-002: Filtrar por sucursal "Centro" → Debe retornar 100 productos

CP-003: Filtros combinados (categoría + color) → Resultados de intersección

CP-004: Filtros sin resultados → Mensaje "0 productos encontrados", "No se encontraron artículos que coincidan con los filtros seleccionados."

CP-005: Rendimiento → Consultas ejecutadas en < 2 segundos

4.2 Pruebas del CU-002 (Carrito persistente):

CP-006: Agregar producto al carrito vacío → Carrito con 1 ítem, al volver a iniciar sesión sigue el ítem en el carrito

CP-007: Eliminar producto del carrito → Actualizar total y lista de ítems

CP-010: Persistencia entre sesiones → Carrito mantiene ítems al recargar página

CP-011: Diferentes visitantes → Carritos independientes por cookie/session

4.3 Pruebas de autenticación:

CP-012: Login con credenciales válidas → Token JWT generado, redirección correcta

CP-013: Login con credenciales inválidas → Error 401, "Credenciales Inválidas. Acceso Denegado."

CP-014: Registro de usuario o administrador nuevo → Usuario creado, campos validados

CP-015: Registro con email duplicado → Error 400, "Fallo en el registro: Email ya registrado!"

CP-016: Acceso a rutas protegidas → mensaje "Acceso no autorizado. Debe iniciar sesión como administrador."

CP-017: Logout → Invalidación de token, limpieza de sesión

4.4 Pruebas de integración:

CP-018: Flujo completo: navegar → filtrar → agregar al carrito → login → checkout

CP-019: Gestión de inventario: stock actualizado tras operaciones

5. Indexación y optimización (Opcional)

Índices aplicados en este proyecto:

// índices para productos

- `db.productos.createIndex(
 { "inventario.sucursal_id": 1, "inventario.stock": 1 },
 { name: "idx_inventario_sucursal_stock" }
);`

Uso: consultas que filtran por inventario de una sucursal y/o por stock (ej.: buscar productos en sucursal X con stock>0).

Beneficio: evita escanear todos los productos y sus arrays, el índice permite localizar rápidamente documentos donde exista un elemento inventario con ese sucursal_id y stock.

- `db.productos.createIndex({ "categoria": 1 });`

Uso: búsquedas por categoría (`find({ categoria: "camisas" })`).

Beneficio: reduce drásticamente la cantidad de documentos leídos al restringir por categoría.

- `db.productos.createIndex({ "atributos.color": 1 });`

Uso: búsquedas rápidas por color en la UI (ej. “mostrar sólo Azules”).

Beneficio: la consulta usa el índice y devuelve resultados al instante en vez de leer toda la colección, así la página carga mucho más rápido.

- `db.productos.createIndex({ "precio": 1 });`

Uso: ordenamientos y rangos por precio (`sort({ precio: 1 })`, `find({ precio: { $gte: X } })`).

Beneficio: permite órdenes y búsquedas por rango sin hacer sort en memoria y con menos documentos escaneados.

// Índices para carritos

- `db.carritos.createIndex({ "visitanteld": 1 });`

Uso: recuperar el carrito de un visitante rápido (lookup por visitanteld).

Beneficio: permite encontrar el carrito de un visitante al instante —en vez de revisar todos los documentos, el índice va directo al registro, haciendo la búsqueda mucho más rápida y ligera.

- `db.carritos.createIndex(
 { "ultimaActividad": 1 },
 { expireAfterSeconds: 2592000 } // TTL 30 días
);`

Uso: TTL para borrar carritos inactivos automáticamente (30 días en el script).

Beneficio: automatiza limpieza de datos antiguos, conserva espacio y estabilidad.

// Índices para usuarios

- `db.usuarios.createIndex({ "email": 1 }, { unique: true });`

Uso: login/registro/lookup por email.

Beneficio: búsquedas por email son instantáneas y la restricción UNIQUE garantiza integridad (no hay duplicados de email), además de forzar un índice que acelera autenticación.

// Índices para órdenes

- `db.ordenes.createIndex({ "usuariold": 1 });`

Uso: consultar órdenes de un usuario.

Beneficio: reduce documentos leídos al filtrar por usuariold.

- `db.ordenes.createIndex({ "estado": 1 });`

Uso: filtrar órdenes por estado (pendiente, enviado...).

Beneficio: rápida agrupación/filtrado en dashboards/admin.

- `dx_orden_sucursal_id: { sucursalId: 1 }`

Uso: consultas por sucursal (p. ej. órdenes en una sucursal específica).

Beneficio: Permite sacar todas las órdenes de una sucursal al instante, en vez de revisar todo el catálogo de pedidos, la base salta directo a los que importan y la consulta va de segundos a casi instantánea.

// índice para sucursal

- `idx_sucursal_nombre: { nombre: 1 }`

Uso: búsquedas por nombre de sucursal.

Beneficio: lookup rápido por nombre de sucursal.

MongoDB usa dos formas principales para mejorar el rendimiento y mantener la base de datos estable: **sharding y replicación**.

- **Sharding** divide los datos en partes más pequeñas y las reparte entre varios servidores. Así se aprovecha mejor la capacidad de cada uno y se puede manejar más información sin que el sistema se vuelva lento.
- **Replicación** crea copias de la base de datos en distintos servidores. Si uno falla, otro puede tomar su lugar, lo que ayuda a evitar caídas y pérdida de datos.

6. Documentación y entrega final

1. Documentación:

Cumplimiento de objetivos:

- ✓ **CU-001 (Filtros avanzados):** Implementado completamente con performance óptimo
- ✓ **CU-002 (Carrito persistente):** Funcionando con operaciones atómicas y TTL
- ✓ **Rendimiento:** Objetivo de < 2 segundos conseguido consistentemente
- ✓ **Flexibilidad:** El esquema permite nuevas categorías y atributos sin cambios estructurales
- ✓ **Escalabilidad:** El diseño está preparado para crecimiento horizontal
- ✓ **Usabilidad:** Interfaz intuitiva con filtros responsivos

Ejemplo de payload de registro:

```
{
  "email": "admin@example.com",
  "password": "Secreto123!",
  "rol": "ROLE_ADMIN",
  "nombre": "Admin"
}
```

2. Reflexión sobre si MongoDB fue la mejor solución

Ventajas observadas en el caso:

- Modelo documental encaja bien con productos que contienen arrays (inventario por sucursal) y atributos variables (color, talla).
- Rápida iteración en desarrollo y flexibilidad de esquema.
- Buen soporte para agregaciones complejas necesarias en filtros avanzados.

Desafíos encontrados:

- Necesidad de mantener coherencia entre la serialización Java y validadores JSON Schema en MongoDB (causó el error original con rol).
- Migraciones de datos (documentos históricos guardados con otro formato) requieren scripts y coordinación.
- Para relaciones fuertemente normalizadas o para garantías ACID en operaciones multi-documento en masa, un RDBMS puede ser más apropiado.

Para la realidad planteada (tienda con productos, inventario por sucursal y filtros avanzados), MongoDB es una buena elección por su modelado flexible y poder del aggregation pipeline. Sin embargo, hay que manejar con cuidado validadores y conversiones (document schema vs. drivers).

Si la aplicación necesitara transacciones multi-documento complejas de forma intensiva o integridad referencial estricta, se debería evaluar una base relacional o un diseño híbrido.